

Cloud Provider Analytics

72.80 - Big Data

Bloise, Luca - 63004

Scheffer, Tomás - 63393

Tepedino, Cristian - 62830



Indice

01

Introducción

Descripción del problema a resolver

02

Arquitectura

Diseño de la solución a alto nivel

03

DataLake

Detalles de implementación de cada sección

04

Serving

Muestra de queries en Cassandra/AstraDB

05

Demo

Prueba en vivo del funcionamiento

01 ✨ Introducción ✨



Contexto

Una empresa proveedora de servicios cloud necesita centralizar y analizar la información generada por sus clientes para mejorar la toma de decisiones en distintas áreas del negocio.

El objetivo es construir un pipeline de datos que procese información proveniente de múltiples fuentes para generar indicadores destinados a:

- **FinOps:** monitoreo de costos y consumo.
- **Soporte:** seguimiento de tickets, SLA y satisfacción.
- **Producto:** análisis del uso de servicios y métricas de GenAI.

La solución debe combinar procesamiento batch para datos maestros e históricos con procesamiento near real-time para eventos de uso.



Consideraciones

La solución implementada debía cumplir los siguientes requisitos:

- Garantizar la calidad y consistencia de la información.
- Procesar datos históricos y en tiempo casi real.
- Soportar la evolución del modelo de datos sin interrumpir el servicio.
- Asegurar confiabilidad mediante procesamiento idempotente.
- Escalar para grandes volúmenes de información.
- Publicar datos listos para análisis y visualización.



Objetivo - Armar un Datalake

Construir un pipeline de datos con PySpark

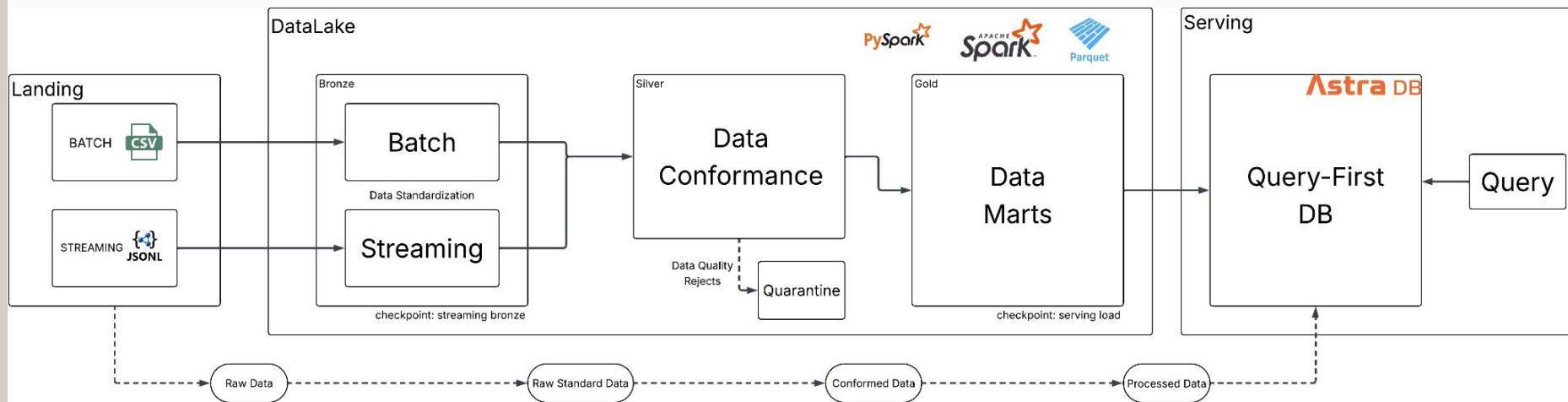
Usar Parquet como storage intermedio gestionado por Spark

Publicar marts de analítica en AstraDB que luego puedan ser consultados por herramientas de visualización



02 ✨ Arquitectura ✨

Arquitectura



Tipo de arquitectura elegida: Lambda (batch y streaming separados)



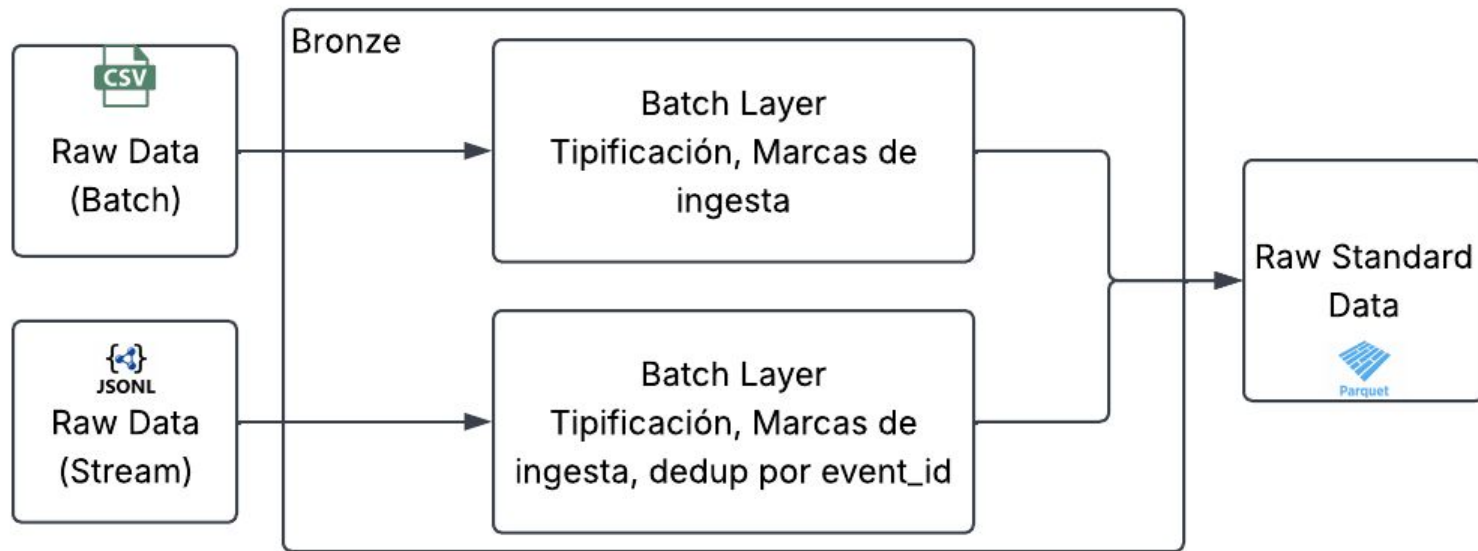
03 ✨ Datalake ✨

Datalake/Landing

Dataset	Uso	Campos
customers_orgs.csv	CRM	org_id, org_name, industry, hq_region, plan_tier, is_enterprise, signup_date, sales_rep, lifecycle_stage, marketing_source, nps_score
users.csv	Usuarios	user_id, org_id, email, role, active, created_at, last_login
resources.csv	Inventario Cloud	resource_id, org_id, service, region, created_at, state, tags_json
billing_monthly.csv	Facturación	invoice_id, org_id, month, subtotal, credits, taxes, currency, exchange_rate_to_usd
support_tickets.csv	Soporte	ticket_id, org_id, category, severity, created_at, resolved_at, csat, sla_breached
marketing_touches.csv	Marketing	touch_id, org_id, campaign, channel, timestamp, clicked, converted
nps_surveys.csv	NPS	org_id, survey_date, nps_score, comment
usage_events_stream/*.jsonl	Telemetría	event_id, timestamp, org_id, resource_id, service, region, metric, value, unit, cost_usd_increment, schema_version, carbon_kg, genai_tokens*

Datalake/Bronze

Mismo grano que la fuente pero con tipificación explícita, dedupe por **event_id** para streams, marcas de ingesta **ingest_ts**, **source_file**. Se guardan en formato parquet.



Datalake/Bronze

Reglas de calidad implementadas:

Batch

- Validación de tipos de datos.
- Dedupe por clave natural antes de escribir (idempotencia)

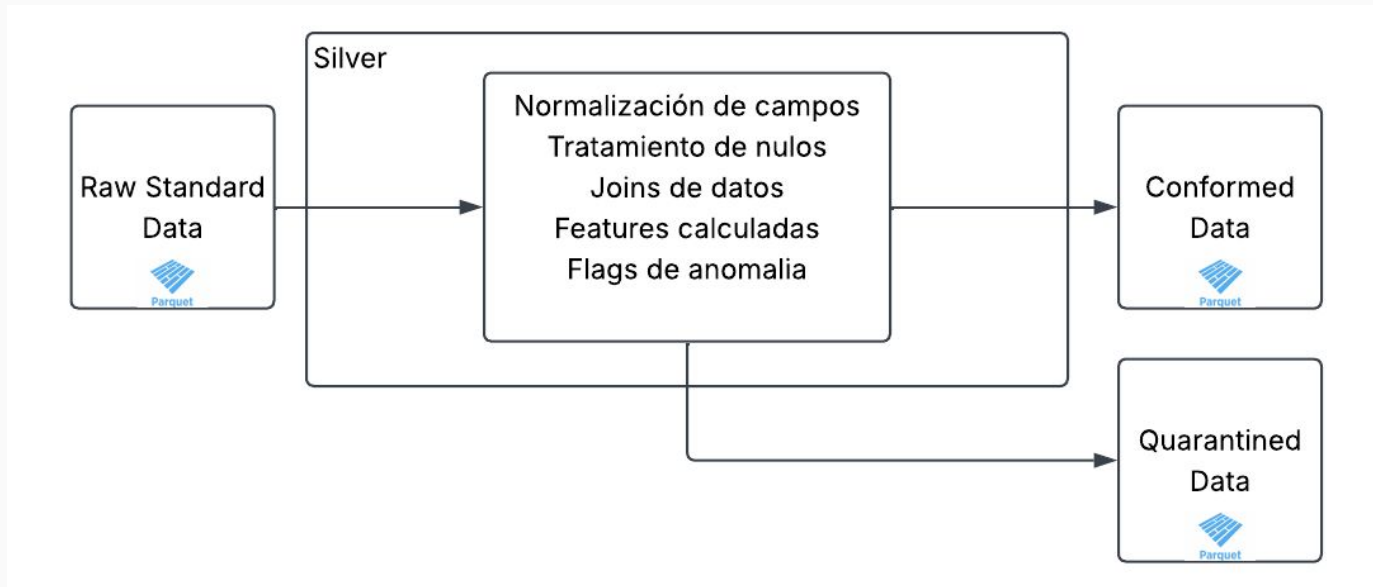
Streaming

- Validación de tipos.
- Validación de campos obligatorios.
- Compatibilidad entre schema v1 y v2 (carbon_kg, genai_tokens).
- Flag `is_late_arrival` si la latencia es de más de 10 minutos.
- Eliminación de duplicados por `event_id` utilizando una ventana temporal (2hrs).

Datalake/Silver

Datos conformados: normalización, joins con dimensiones (customers_orgs, resources, users), features calculadas y flags de anomalía.

Se aíslan errores en quarantine junto con el campo adicional error_reason.



Datalake/Silver

Datasets Generados	Campos principales	Propósito
customers_orgs	org_id, industry, hq_region, plan_tier, nps_score	Dimensión de clientes
users	user_id, org_id, role, active	Dimensión de usuarios
resources	resource_id, org_id, service, region	Dimensión de recursos
usage_events	event_id, org_id, service, cost_usd_increment, requests, cpu_hours, genai_tokens, carbon_kg, is_cost_anomaly	Eventos enriquecidos
org_service_daily	org_id, usage_date, service, daily_cost_usd, requests, cpu_hours, storage_gb_hours, genai_tokens, carbon_kg	Agregación diaria por organización y servicio
quarantine	Filas rechazadas + error_reason	Aislamiento de errores

Datalake/Silver

Condiciones en las cuales un dato cae en quarantine:

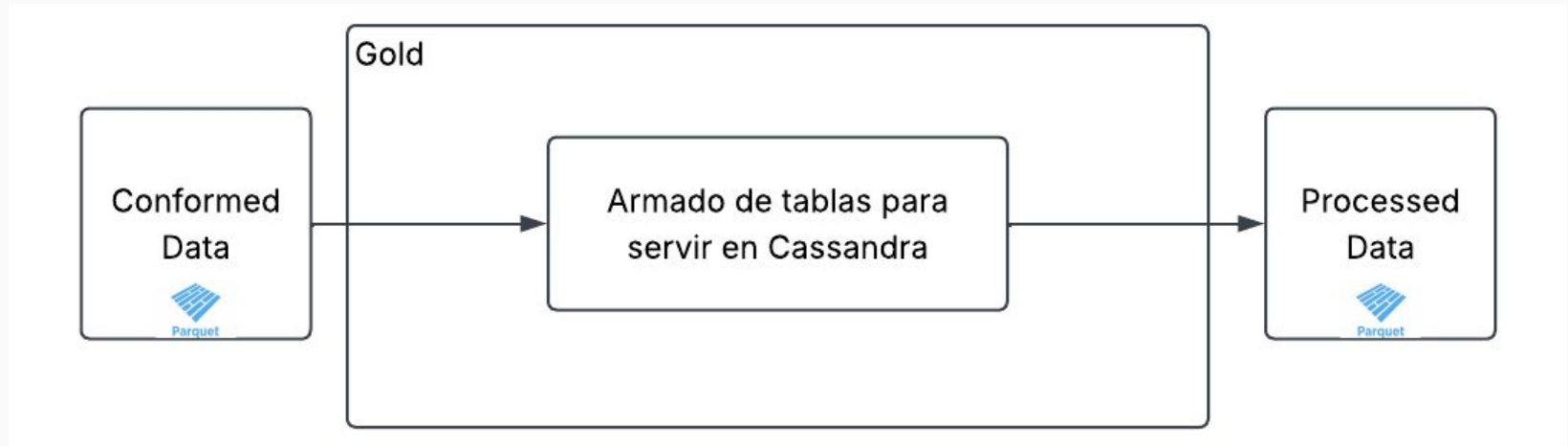
- Integridad: event_id nulo/duplicado; event_ts inválido o futuro
- Conformance: value con unit nulo; cost_usd_increment < -0.01
- Referencial: org_id o resource_id huérfanos
- Catálogo: service / region fuera de lista permitida
- Umbral máximo de late data: latencia > 30 min
- Maestros: PK nula u org_id inexistente

Condiciones en las cuales un dato se identifica como anomalía:

- $|z\text{-score}| > 3$ por (org_id, service)
- $|z\text{ MAD}| > 3.5$
- Costo fuera de percentiles P1/P99
- Costo > $p99 \times 2$

Datalake/Gold

Vistas/tables orientadas a analítica (FinOps, Soporte, Producto/Usage), listas para servir en Cassandra (AstraDB).



Datalake/Gold

Mart	Objetivo
org_daily_usage_by_service	Análisis FinOps
org_top_services_by_cost	Top-N costo 14 días
revenue_by_org_month	Facturación
cost_anomaly_mart	Detección de anomalías
tickets_by_org_date	KPIs de soporte
genai_tokens_by_org_date	Consumo GenAI
nps_by_org_date	CRM/NPS
marketing_touches_by_org_channel	Marketing



04 ✨

Serving ✨

Diseño de Tablas

Astra DB



org_daily_usage_by_service

Partition key: org_id

Sort key: usage_date DESC, service ASC

org_top_services_by_cost

Partition key: org_id, period_end

Sort key: rank ASC, service ASC

tickets_by_org_date

Partition key: org_id, severity

Sort key: usage_date DESC, service ASC

revenue_by_org_month

Partition key: org_id

Sort key: billing_month DESC

genai_tokens_by_org_date

Partition key: org_id

Sort key: usage_date DESC

Query 1

Costos y requests diarios por org y servicio en un rango de fechas

```
SELECT
    usage_date,
    service,
    total_daily_cost_usd,
    total_requests
FROM org_daily_usage_by_service
WHERE org_id = 'org_rixa11dp'
    AND usage_date >= '2025-07-01'
    AND usage_date <= '2025-08-31';
```

Query 2

Top-N servicios por costo acumulado en los últimos 14 días para una organización

```
SELECT
    service,
    accumulated_cost_usd,
    period_start,
    period_end,
    rank
FROM org_top_services_by_cost
WHERE org_id = 'org_rixalldp'
    AND period_end = '2025-08-31'
ORDER BY rank ASC
LIMIT 4;
```

Query 3

Evolución de tickets críticos y tasa de SLA breach por día (últimos 30 días)

```
SELECT
    ticket_date,
    ticket_count,
    sla_breach_count,
    sla_breach_rate,
    avg_csat
FROM tickets_by_org_date
WHERE org_id = 'org_rixa11dp'
    AND severity = 'high'
    AND ticket_date >= '2025-08-02'
    AND ticket_date <= '2025-08-31';
```

Query 4

Revenue mensual con créditos/impuestos aplicados (normalizado a USD)

```
SELECT
    billing_month,
    subtotal_usd,
    credits_usd,
    taxes_usd,
    revenue_usd,
    invoice_count
FROM revenue_by_org_month
WHERE org_id = 'org_rixalldp'
    AND billing_month >= '2025-06-01'
    AND billing_month <= '2025-08-01';
```

Query 5

Tokens GenAI y costo estimado por día (si existen)

```
SELECT
    usage_date,
    total_genai_tokens,
    estimated_cost_usd
FROM genai_tokens_by_org_date
WHERE org_id = 'org_rixa11dp'
    AND usage_date >= '2025-08-01'
    AND usage_date <= '2025-08-31';
```


05 ✨

Demo ✨



Muchas Gracias!



CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

